A map you bring.
A world you can navigate.

Product vision and technical planning document for a cross-platform, offline-first trekking app that reconstructs user-provided maps while preserving their original visual identity.

Core concept
Photograph or import almost any map. Reconstruct it as an interactive digital map in the same visual style, then show the trekker's live GPS position and recorded movement on it. Keep the original image as a calibrated fallback.

Platforms: iOS and Android

Initial experience: 2D map reconstruction and offline GPS tracking

Later extension: Optional 3D terrain visualisation

Status: Planning only — no implementation decisions are final

Prepared as a working product brief. It distinguishes the agreed vision from assumptions, technical constraints and decisions that still need to be made.

Contents

- 1. Executive summary
- 2. Product vision and principles
- 3. Problem statement and target users
- 4. Core experience and user journeys
- 5. Functional requirements
- 6. Map reconstruction: the central technical challenge
- 7. Spatial calibration and GPS alignment
- 8. Offline-first system architecture
- 9. Data model and local storage
- 10. User interface and interaction design
- 11. iOS and Android considerations
- 12. Performance, battery and reliability
- 13. Safety, privacy and security
- 14. Testing and validation strategy
- 15. MVP definition and roadmap
- 16. Risks and mitigations
- 17. Success metrics
- 18. Open questions and decisions
- 19. Glossary

This document is intentionally comprehensive. It is a planning baseline, not a claim that arbitrary map reconstruction or perfect navigation accuracy has already been solved.

1. Executive summary

The proposed app is an offline-first trekking companion built around a distinctive idea: the user brings the map, and the app turns that map into an interactive digital experience without replacing its original visual identity.

The user photographs or imports a map. The app analyses its visual content and builds a digital representation that can include trails, rivers, contour lines, mountains, labels, landmarks, campsites and other map elements. Unlike conventional mapping apps, it should not force every map into a standard cartographic style. A traditional topographic map should still look like a traditional topographic map; an illustrated or cartoon treasure map should retain its colours, textures, symbols and visual theme.

Once the user marks their starting point and the map is calibrated as far as the available information permits, GPS updates the user's position on the reconstructed map. The app records movement, can show the track already travelled, and keeps the map and trek data on the device so navigation can continue without mobile data.

A calibrated overlay on the original photograph is retained as a failsafe. The primary experience is the reconstructed digital map; the original image remains accessible if the reconstruction is incomplete, uncertain or visually wrong. 3D terrain is a later phase, not a requirement for the first release.

> **Product thesis: make a user-supplied map usable as a live, offline navigation surface while preserving the map's own character. The differentiator is not merely GPS tracking; it is style-preserving map digitisation paired with a trustworthy spatial model.**

What the product is

- A cross-platform mobile app for iPhone and Android.
- A map capture/import, reconstruction, calibration and navigation workflow.
- An offline-first system whose essential navigation features do not depend on a remote service.
- A visual reconstruction system with a separate internal representation for geometry, features and coordinates.
- A 2D-first product with a possible future 3D view.

What the product is not

- Not simply a camera overlay on a paper map.
- Not a replacement for all global map providers or a guarantee of turn-by-turn routing on arbitrary maps.
- Not a promise that every arbitrary image can be reconstructed perfectly without user review.
- Not a substitute for a compass, proper outdoor preparation or established safety practices.

2. Product vision and principles

Vision statement

A trekker can take the map they already trust, convert it into a digital map that still looks like itself, and see their real-world movement on it—without needing internet access during the trek.

Design principles

Principle	Meaning for product decisions
Preserve the source identity	Retain the map's palette, texture, typography, symbols, line styles, illustration language and overall composition wherever possible.
Structure beneath the surface	The app should understand geometry and feature types independently of how they are drawn.
Offline by default	The map, calibration, navigation state and recorded track must be stored locally and remain usable without connectivity.
Calibration before confidence	A location marker should only be as confident as the map's geographic alignment and GPS conditions allow.
Make uncertainty visible	When the app is unsure whether a line is a trail or river, or when georeferencing is weak, it should ask the user or clearly communicate uncertainty.
Keep a fallback	The original map image and its GPS/track overlay should remain available.
User control	The user chooses when to create a map, start tracking, stop tracking, export or delete trek data.
Build 2D first	Prove reconstruction, calibration, offline tracking and reliability before adding 3D.

The most important architectural separation

The map should have at least two related but separate representations:

- **Visual representation:** what the user sees—textures, colours, labels, hand-drawn features, icons and theme.
- **Spatial/semantic representation:** the geometry and meaning the app uses—paths, points, polygons, contours, possible elevation values, labels, confidence scores and coordinate transforms.

For example, a river can be stored as a polyline while being rendered as a hand-painted blue ribbon. A trail can be stored as a path while retaining the source map's dotted red styling. This avoids forcing the internal data model to dictate the visual style.

Product quality hierarchy

1. Navigation correctness and safety-relevant honesty.
2. Preservation of the source map's appearance.
3. Offline reliability and recoverability.
4. Usability and speed of the capture-to-navigation workflow.
5. Advanced visualisation such as 3D.

3. Problem statement and target users

Problem

Trekkers may rely on printed maps, downloaded images, guidebook maps or maps made for a specific trail. Conventional GPS apps usually display their own base map rather than the exact map a trekker has. A photograph of a paper map is easy to carry but does not automatically show the user's live position. Generic map conversion can lose the source map's visual character, and remote areas may have limited or no data connectivity.

Primary user groups

User group	Need	Likely challenge
Independent trekker	See live location on a map they already have.	Map may be a photo or scan with no explicit coordinates.
Backpacker / multi-day hiker	Offline navigation and track history across long trips.	Battery, storage and reliable background tracking.
Map enthusiast	Preserve an illustrated or specialised map's aesthetic.	Style must survive conversion rather than become generic.
Guide / group leader	Prepare a map before a trip and inspect movement afterwards.	Calibration, sharing/export and understandable confidence.
Casual outdoor explorer	Simple setup without GIS expertise.	Need a guided workflow and minimal technical terminology.

Representative use cases

- Photograph a folded paper trail map at the trailhead and use it for a day hike.
- Import a scanned map from a guidebook, calibrate it and track a multi-day trek.
- Use a hand-drawn or illustrated map for a themed outdoor activity while retaining its visual style.
- Review the path already travelled after losing sight of a trail junction.
- Continue navigating when mobile data disappears.
- Open the original image when the reconstructed version omits or misclassifies a detail.

4. Core experience and user journeys

Journey A: create a map

1. Open the app and choose **Capture map** or **Import image/PDF page**.
2. Capture the map under useful lighting, or crop/rotate an imported image.
3. The app corrects perspective and image distortion where possible, while preserving the untouched original.
4. The app detects likely text, lines, symbols, regions, labels and visual styles.
5. The user sees a preview of the reconstructed map and can inspect uncertain elements.
6. The user confirms the map name, optional notes and whether to store a local copy of the source image.
7. The app asks for calibration: known coordinates/grid, scale and orientation information, reference points, or a guided best-effort method if the map has no geographic references.
8. The map is saved locally and can be reopened without internet.

Journey B: start a trek

1. Open a saved map and select **Start trek**.
2. Choose or confirm the starting point on the map.
3. If calibration is not complete, the app explains what is missing and offers the best available calibration method.
4. The app checks location permission, GPS availability and estimated location quality.
5. Tracking starts only after clear user confirmation.
6. The current position appears on the reconstructed map, and the travelled path is recorded.
7. The user can switch to the original-image fallback at any time.

Journey C: finish and review

1. Tap **Finish trek** to stop recording.
2. Show duration, distance travelled, track and any location-quality warnings.
3. Save the trek locally with the map it used.
4. Offer export/share options only after the user explicitly chooses them.
5. Allow the user to rename, archive or delete the trek.

Journey D: reconstruction is incomplete

- Display the original map image alongside or behind the reconstructed view.
- Mark uncertain features in a review interface before the user relies on them.
- Allow corrections such as 'this is a trail', 'this is a river', 'move this landmark' or 'do not include this feature'.
- Keep corrections local initially; no upload should be required for the map to become usable.
- Do not silently invent missing paths, labels or terrain.

5. Functional requirements

5.1 Capture and import

- Support camera capture with framing guidance and a review step.
- Support importing common image formats; evaluate PDF page import for the initial release or a follow-up.
- Keep the original source immutable so later reconstruction changes can be compared against it.
- Support crop, rotate and perspective correction without overwriting the original.
- Detect blur, glare, low contrast, shadows, folds and incomplete map boundaries; suggest retaking only when materially useful.
- Handle large images without blocking the interface; show progress and allow cancellation.

5.2 Reconstruction and visual fidelity

- Produce a digital 2D representation with the source map’s visual language.
- Preserve colour relationships, textures, labels, iconography, line styles and overall composition.
- Retain source pixels or textured regions when redrawing would reduce fidelity.
- Represent detectable features as editable objects beneath the rendered visual surface.
- Provide confidence or review status for detected features.
- Support a user correction workflow and preserve corrections across app restarts.
- Never treat an unverified detected line as a confirmed navigable trail.

5.3 Calibration

- Offer coordinate-grid or known-coordinate calibration when available.
- Offer scale and orientation inputs when they are printed on the map.
- Allow the user to mark multiple reference points and associate them with known geographic positions where possible.
- Explain that a single start point anchors one position but does not independently establish map scale, rotation and geographic distortion.
- Store the calibration method, transform, residual/error estimate and confidence.
- Let the user recalibrate if the initial alignment is visibly wrong.
- Keep image-space navigation available as a limited fallback when true geographic alignment cannot be established.

5.4 Navigation and tracking

- Show the live GPS position and heading where device sensors support it.
- Record a breadcrumb track with timestamps and location-quality metadata.
- Calculate distance travelled and elapsed time, with appropriate smoothing and clear distinction between measured and estimated values.
- Support pause/resume and explicit stop/finish controls.
- Handle app backgrounding, screen lock, interruptions and process termination as reliably as the platform permits.
- Show last known position and its age if location updates stop.
- Indicate poor GPS conditions and avoid implying that the dot is exact.
- Do not promise turn-by-turn routing unless a trustworthy route graph and route-validation system are separately designed.

5.5 Original-image failsafe

- Retain a view of the original source image.
- Draw the GPS position and recorded track on the original image using the same spatial calibration where valid.
- Allow quick switching between reconstructed and original views.
- Clearly indicate when the fallback alignment is approximate or only image-space based.
- Ensure the fallback is available offline and does not depend on a cloud copy.

5.6 Offline and local management

- Save map image, reconstruction data, calibration and trek data locally.
- Allow opening and using a completed map in airplane mode.
- Persist work progressively so a crash does not discard the entire reconstruction.
- Provide storage usage, delete-map and delete-trek controls.
- Support export as an optional user action; no automatic upload is required.
- Consider encrypted app storage and device backup behaviour during implementation.

6. Map reconstruction: the central technical challenge

The defining technical challenge is not drawing a map-like screen. It is turning an arbitrary visual source into a structured, faithful and useful digital representation. Source maps vary widely: some have clear grids and legends, while others are artistic, low-resolution, folded, handwritten or designed only for visual storytelling.

6.1 Suggested processing pipeline

Stage	Purpose	Output
1. Preserve source	Store the untouched input and basic metadata.	Original image asset
2. Image preparation	Crop, rotate, correct perspective, improve contrast where useful.	Working image plus transform history
3. Visual analysis	Estimate palette, regions, text blocks, lines, symbols and textures.	Image feature candidates
4. Text and symbol detection	Read labels and identify likely map symbols.	Labels and symbol candidates with confidence
5. Geometry extraction	Trace lines, boundaries, point symbols and contour-like features.	Vector candidates
6. Semantic classification	Estimate whether candidates are trails, rivers, contours, boundaries or decorative art.	Typed features with confidence
7. Style reconstruction	Render extracted features in the original visual style; retain source textures where appropriate.	Layered digital map
8. Human review	Let the user correct uncertain or important elements.	Reviewed feature set
9. Calibration	Relate map positions to geographic positions where possible.	Coordinate transform and confidence
10. Local packaging	Persist everything needed for offline viewing and tracking.	Offline map bundle

6.2 Conventional algorithms versus machine learning

A sophisticated program can do a large amount without a generative AI system. Image processing, OCR, connected-component analysis, edge detection, line tracing, colour clustering, geometric transforms and deterministic rendering are useful foundations. Local machine-learning models may help classify difficult symbols or distinguish features when rules are insufficient.

A generative model should not be allowed to invent geographic detail. It might produce a visually plausible but incorrect trail, move a river or alter a label. For a navigation product, plausible-looking errors are especially dangerous. The preferred design is evidence-based extraction plus deterministic rendering, with optional ML assistance and explicit user review.

6.3 Style-preserving rendering

- Maintain a style palette extracted from the source: colours, stroke widths, line patterns, textures and text treatments.
- Keep a mapping between each digital object and the source region(s) that support it.
- Use original pixels or image patches as texture layers for complex illustration where vector recreation would be costly or inaccurate.
- Use vector geometry for features that benefit from interaction, scaling and spatial queries.
- Use separate visual layers for background texture, terrain art, text, detected features, user annotations, GPS dot and track.
- Make the GPS marker visually legible without changing the underlying map theme.

6.4 Why a hybrid reconstruction is likely

Pure vector redraw risks losing artistic texture and detail. Pure image overlay does not provide structured, editable features. A hybrid system can preserve complex source art as raster textures while tracing important features into vectors. The app can therefore keep the map’s appearance while making selected objects interactive.

6.5 Confidence and review

Every detected feature should have a status such as candidate, user-confirmed or rejected. Confidence is an engineering aid, not a guarantee of correctness. High-impact features—especially trails, crossings, cliff symbols and junctions—should be reviewable. The app should not make an unverified feature appear more authoritative than the source.

6.6 Limits to acknowledge

- Low-resolution, blurry or occluded maps may not contain enough detail to extract all features.
- Legends vary; the same colour or line style can mean different things across maps.
- Decorative illustrations can resemble geographic features.
- Hand-drawn maps may not be to scale or geographically accurate.
- Some maps distort distances intentionally for readability.
- Text recognition can confuse similar characters or labels.
- Reconstruction speed and fidelity will depend on map complexity and device capability.

7. Spatial calibration and GPS alignment

> **Critical constraint: marking the user's starting point on the map is useful, but one point alone generally cannot determine geographic scale, rotation, projection or distortion. The product must provide additional calibration options or clearly limit what it claims.**

7.1 Coordinate spaces

The app should distinguish at least three coordinate spaces:

- **Image space:** pixel coordinates in the original photograph or scan.
- **Reconstructed-map space:** coordinates used by the digital rendering and its feature geometry.
- **Geographic space:** latitude/longitude or an appropriate local coordinate reference used to associate the map with GPS.

A stored transformation links these spaces. It should be possible to transform a GPS location to the corresponding position on the reconstructed map and to the original image for the failsafe.

7.2 Calibration methods, in preferred order

Method	What user provides	Strength / limitation
Known grid or coordinates	Map grid intersections or printed coordinate labels.	Strong when the map is trustworthy and enough references are available.
Multiple known landmarks	Several points with known real-world coordinates.	Can estimate alignment and reveal error; the landmarks must be correctly identified.
Scale plus orientation plus anchor	Printed scale, north/orientation and a known starting point.	Useful for some maps, but projection and local distortion can still matter.
Reference map alignment	User aligns the source with a known geographic layer, if available offline.	Potentially useful but introduces dependency on reference data and licensing.
Single-point anchor only	One current position marked on the map.	Anchors one point but cannot establish full map scale and orientation on its own.
Image-space-only mode	User marks the starting location and accepts approximate tracking.	Can provide a visual fallback, but should not be presented as reliable geographic navigation.

7.3 Calibration workflow

1. Ask whether the map has a coordinate grid, scale bar, north arrow or labelled coordinates.
2. Prefer the simplest reliable method supported by the source.
3. Where possible, ask for multiple reference points spread across the map rather than clustered together.
4. Estimate alignment quality and show a simple status such as Good, Approximate or Insufficient data.
5. Allow the user to preview the GPS marker on the map before starting the trek.
6. Save the calibration inputs and method so the same map can be reused.
7. Recalibrate if the user changes the crop or image transform.

7.4 GPS limitations

- GPS may work without mobile data, but accuracy varies with terrain, tree cover, weather and device hardware.
- Mountain walls and narrow valleys can block or reflect satellite signals.
- GPS noise can make the dot jump or make recorded distance appear longer than actual travel.
- Compass heading can be unreliable near metal objects or when the user is stationary.
- The UI should show accuracy radius or quality status where practical, not just a precise-looking dot.
- Track smoothing should not erase meaningful switchbacks or falsely snap users to an unverified trail.

7.5 What the app can honestly promise

When calibration is strong and GPS conditions are good, the app can show a useful live position on the source map. When either is weak, it should show the uncertainty and keep the original map accessible. It should not claim that a start-point tap alone makes any map geographically accurate.

8. Offline-first system architecture

A suitable high-level architecture separates the mobile interface, reconstruction pipeline, map model, spatial engine, GPS service and local persistence. These can be modules within one mobile app; they do not require a server.

Module	Responsibilities	Offline requirement
Mobile UI	Capture/import, review, calibration, map viewing, trek controls.	Fully available offline.
Image pipeline	Crop, rectify, normalise and prepare images.	Runs on device.
Reconstruction engine	Detect text/features, create map layers, assign confidence.	Core path runs on device.
Map model	Store visual layers, geometry, labels, feature metadata and style.	Local, versioned representation.
Spatial engine	Transform image/map/GPS coordinates and calculate spatial relationships.	Local calculations.
Location service	Request location updates and manage trek state.	Uses device sensors; no network required for core GPS.
Renderer	Draw map layers, GPS dot, track and selected features.	Local rendering.
Storage layer	Persist images, map packages, calibration, settings and tracks.	Local database/files.
Optional sync/export	User-initiated backup, sharing or export in a future phase.	Never required for navigation.

8.1 Data flow

Capture/import → image preparation → feature extraction → style-preserving reconstruction → user review → calibration → local map package → GPS tracking and rendering. The original image remains attached to the map package throughout.

8.2 Cross-platform strategy

The product must target both iOS and Android. A shared UI and shared core logic may reduce duplicated work, but camera, location permissions, background execution, file import and some image-processing capabilities are platform-specific. The implementation choice should be made after a small technical prototype, not by assuming one framework automatically solves all native requirements.

- Evaluate a shared cross-platform framework for common UI and product logic.
- Keep platform-specific adapters for location, camera, permissions, background operation and file access.
- Benchmark image processing on representative lower-, mid- and high-tier devices.
- Use the same reconstruction and map-data specification across platforms so a map package has consistent behaviour.
- Do not commit to a specific stack until performance, plugin reliability and background tracking are tested.

8.3 Local processing strategy

The default should be on-device processing. Optional cloud processing, if ever introduced, must be opt-in, clearly explained and never required to navigate. This supports privacy, offline use and predictability. It also means the reconstruction pipeline must be designed to fit device memory, CPU/GPU and thermal limits.

9. Data model and local storage

The data model should preserve source evidence, reconstructed features and geographic alignment independently. The following is conceptual, not a final database schema.

Entity	Key information
MapProject	Unique ID, title, creation date, source type, status, reconstruction version, last opened.
SourceAsset	Original image/PDF page, dimensions, checksum, orientation, crop and perspective transforms.
MapLayer	Layer order, visibility, rendering mode, style metadata, source asset references.
MapFeature	Feature ID, type candidate, geometry, label, confidence, review state, source region, style reference.
Calibration	Method, reference points, transform, scale/orientation inputs, coordinate reference assumptions, estimated residuals and confidence.
TrekSession	Start/end times, associated map, state, tracking configuration and user notes.
TrackPoint	Latitude/longitude, timestamp, reported accuracy, optional altitude/heading/speed, validity flags.
UserCorrection	Feature edit, prior value, new value, timestamp and source map version.
AppSettings	Tracking preferences, privacy choices, display preferences and accessibility settings.

Storage principles

- Keep source image and reconstruction assets linked but independently recoverable.

- Use a versioned map-package format so future reconstruction improvements do not silently corrupt old maps.
- Persist track points in batches and recover safely after process termination.
- Use spatial indexing only where it improves performance; avoid premature complexity.
- Expose storage usage and allow the user to delete map packages and tracks.
- Plan for low-storage situations and interrupted imports.
- Define whether user exports contain raw GPS coordinates and communicate this clearly.

Map package concept

A completed map can be treated as a self-contained local package: source image, corrected working image, rendered layers or textures, feature geometry, style metadata, calibration transform, confidence/review data and optional user notes. It should reopen without needing to rerun every reconstruction stage.

10. User interface and interaction design

Primary screens

Screen	Purpose / essential controls
Home / Maps	Create map, import map, open recent maps, see stored trek sessions and available storage.
Capture	Camera framing, glare/blur guidance, retake and confirm.
Crop & correct	Crop, rotate, perspective correction, reset to original.
Reconstruction preview	Compare source and reconstructed views; inspect uncertain features.
Feature review	Confirm, edit or reject candidate labels, trails, rivers and landmarks.
Calibration	Choose calibration method, place reference points, enter known scale/coordinates, preview alignment.
Map view	Pan/zoom, live position, track, layer toggle, source-image fallback and location-quality status.
Trek controls	Start, pause, resume, finish, elapsed time, distance and tracking status.
Trek summary	Track overview, duration, distance, notes and export/delete options.
Settings & privacy	Location permissions, background tracking explanation, data management and privacy controls.

Interaction guidelines

- Make the first successful map creation achievable without GIS knowledge.
- Use plain language; explain calibration only when needed.
- Show reconstruction progress and allow cancellation without losing the original.
- Distinguish 'map reconstruction complete' from 'geographic calibration complete'.
- Make the current position visually obvious while preserving the map style.
- Keep the fallback one tap away during a trek.
- Use large, high-contrast controls suitable for outdoor use and gloved hands where practical.
- Prevent accidental trek termination; make pause and finish states clear.
- Offer a north-up mode and optional heading-up mode if compass quality permits.

Accessibility and outdoor usability

- Support system text scaling and screen-reader labels.
- Do not rely on colour alone to distinguish GPS state or feature type.
- Provide contrast and marker-size options without forcing a complete map restyle.
- Support dark surroundings and bright daylight through carefully tested contrast choices.
- Keep critical controls discoverable and avoid tiny touch targets.
- Consider haptic feedback for tracking start/stop and selected warnings.

11. iOS and Android considerations

iOS

- Request location permissions in context and explain the benefit before requesting background access.
- Design for platform-specific background-location behaviour, battery constraints and user-visible indicators.
- Test screen lock, app switching, calls, low-power modes and force-quit behaviour.
- Use supported camera/photo-library APIs and handle permission denial gracefully.
- Test a range of iPhone generations, not only the latest hardware.

Android

- Account for runtime location permissions, background-location restrictions and foreground-service requirements where applicable.
- Handle manufacturer-specific battery optimisation that may pause background work.
- Test varied device vendors, Android versions, memory capacities and screen sizes.
- Use supported camera and document-picker flows.
- Make the user-visible tracking state clear where the operating system requires it.

Shared-platform rule

Cross-platform does not mean platform-identical internals. The user experience and map package should be consistent, but native location services, permissions and background execution must follow each operating system’s current rules. Those details need verification against current platform documentation during implementation.

12. Performance, battery and reliability

Performance targets to validate

The figures below are proposed targets for prototype testing, not verified performance guarantees.

Area	Initial target / measurement
Map opening	A saved map should open promptly without rerunning full reconstruction; target a responsive first view within a few seconds on representative devices.
Reconstruction	Measure time by source image size and complexity; offer progress and cancellation.
Pan/zoom	Maintain a responsive map interaction under normal load; profile frame rate on mid-range devices.
GPS update	Update at a configurable interval appropriate for trekking, balancing responsiveness and battery.

Area	Initial target / measurement
Recovery	A forced app restart should recover the saved map and last persisted track points.
Storage	Handle large maps with bounded memory use and clear low-storage behaviour.
Battery	Measure energy use over a representative multi-hour outdoor session on iOS and Android.
Thermals	Avoid running expensive reconstruction continuously during navigation; process before the trek where possible.

Practical engineering choices

- Perform expensive reconstruction before the trek and persist its result.
- Use incremental processing and checkpoints for large maps.
- Downsample preview images while retaining original resolution for detail and fallback.
- Render only visible layers and use caching for expensive textures.
- Separate high-frequency GPS updates from full map reconstruction.
- Make location update frequency configurable or adaptive where platform behaviour permits.
- Do not keep the camera, CPU-intensive image pipeline or unnecessary sensors active during normal tracking.

Reliability and recovery

- Persist the active trek state and track points regularly.
- Handle temporary GPS loss without deleting the last known position or track.
- Show stale-position age and resume updates when available.
- Keep the original map intact even if reconstruction fails.
- Validate map package integrity when opening it.
- Test interruption scenarios: phone call, low battery, app backgrounding, OS termination, device restart and storage exhaustion.

13. Safety, privacy and security

Safety posture

The app supports navigation but cannot guarantee a safe route or a correct position in every environment. A reconstructed map may contain omissions, and GPS can be inaccurate in mountains. The interface should not imply that the app has verified a trail merely because it detected a line in an image.

- Communicate calibration quality and GPS uncertainty.
- Keep the source map accessible.
- Flag unreviewed or low-confidence trail candidates.
- Do not invent bridges, crossings, safe paths or terrain.
- Do not present a straight-line connection as a safe route.
- Encourage users to carry appropriate backup navigation and outdoor safety equipment; do not frame the app as a substitute.

Location permission and background tracking

Background location is sensitive and consumes power. Request it only when the feature genuinely requires it, explain what it enables, and provide a clear way to stop tracking. The product should not require permanent tracking merely to import or view a map.

- Separate permission to use location while the app is open from any background-tracking permission.
- Start recording only after the user explicitly starts a trek.
- Show a persistent, understandable tracking state.
- Stop collection when the trek ends, subject to clearly explained recovery behaviour.
- Let users delete saved track history and maps.
- Never collect or upload location history for analytics by default.

Privacy model

- Local-first storage should be the default.
- Do not upload maps, photos or GPS tracks without explicit consent.
- Keep optional analytics free of precise location data unless a justified, disclosed opt-in is designed.
- Document how backups, exports and OS-level device backups handle map and track data.
- Use secure storage practices and appropriate file/database protections.
- Provide a clear privacy policy before public release.

Security considerations

- Validate imported file types and handle malformed or unusually large files safely.
- Limit memory use and guard against crashes from crafted images.
- Use platform-secure storage options for sensitive metadata where appropriate.
- Do not expose local files through unintended sharing mechanisms.
- Keep dependencies updated and review permissions before release.
- Design exports to be user-initiated and clearly show what data is included.

14. Testing and validation strategy

Build a representative map test corpus

Testing only a few clean topographic maps would create a misleading impression of quality. The corpus should cover a wide range of styles and conditions.

- High-resolution topographic maps with coordinate grids.
- Maps with printed scale bars and north arrows.
- Low-resolution screenshots and compressed images.
- Photographs with perspective distortion, glare, shadows and folded paper.
- Illustrated/cartoon maps with non-standard symbols and stylised trails.
- Hand-drawn maps that may not be to scale.
- Maps with dense labels, overlapping symbols and multilingual text.
- Maps with no coordinate references.
- Maps containing visually similar rivers, roads, trails and contour lines.
- Maps whose geography is distorted for presentation.

Functional test matrix

Test category	Examples
Capture/import	Blurry input, rotate/crop, huge file, denied camera permission, interrupted import.
Reconstruction	Feature precision/recall, text accuracy, style similarity, user correction persistence.
Calibration	Known points, different map rotations/scales, perspective correction, low-confidence alignment.
GPS	Open sky, forest, valley, temporary signal loss, noisy location samples, heading errors.
Offline	Airplane mode after map creation; no network at startup; long trek with no coverage.
Background	Screen lock, app switching, OS process termination, low-power mode, permission changes.
Recovery	Crash during reconstruction, crash during trek, storage exhaustion, device restart.
Cross-platform	Same map package and expected behaviour on supported iOS and Android devices.
Usability	First-time user can create, calibrate and start a trek without specialist knowledge.
Safety	Low confidence is visible; no unverified route is presented as confirmed.

Evaluation metrics for reconstruction

- **Visual fidelity:** human rating of whether the result preserves source appearance, plus image-based similarity metrics where meaningful.
- **Feature detection precision/recall:** measure correct and missed trails, rivers, landmarks and labels against annotated test maps.
- **Geometry error:** compare extracted paths and points to annotated reference geometry.
- **Calibration error:** measure location alignment error against known reference points, separately from GPS error.
- **Task completion:** percentage of users who can create a usable map and start tracking.
- **Correction effort:** time and number of edits required before a map is trusted by the user.

Evaluation caution

A map can look visually convincing while being spatially wrong. Visual fidelity and navigation correctness must be measured independently. The app should never use a high style-similarity score as evidence that its coordinates are accurate.

15. MVP definition and roadmap

Phase 0 — Feasibility prototype

- Select a small, diverse set of map images including topographic and illustrated styles.
- Prototype image correction, source-preserving layers and a basic vector overlay.
- Test map calibration with known reference points and measure alignment error.
- Test GPS-to-map rendering on both iOS and Android.
- Benchmark processing time, memory use and battery impact.

• Make a go/no-go decision based on measured reconstruction quality rather than assumptions.

Phase 1 — MVP: dependable 2D offline navigation

Include	Details
Map capture/import	Capture a map or import an image; crop, rotate and preserve original.
Basic reconstruction	Layered 2D rendering that retains source style; extracted objects where reliable.
Manual correction	User can inspect and correct important detected features.
Calibration	Support robust methods first; disclose limitations when references are insufficient.
Offline map package	Save map, reconstruction and calibration locally.
GPS tracking	Live location, breadcrumb track, pause/resume/finish and basic trek summary.
Original fallback	Source image with calibrated position and track overlay.
Privacy controls	Explicit start/stop tracking, permission explanations and delete controls.
Cross-platform	Supported iOS and Android builds with platform-specific tracking handling.

Phase 1 non-goals

• Perfectly reconstruct every possible map style automatically.

• Guarantee navigation accuracy on maps without enough geographic information.

• Automatic safe-route planning or turn-by-turn directions.

• Cloud-based map sharing, social features or account systems unless later justified.

• 3D terrain reconstruction.

• Continuous tracking when no trek is active.

Phase 2 — Reconstruction quality and usability

• Improve symbol, label and geometry extraction across more map styles.

• Improve user correction tools and uncertainty presentation.

• Support additional import types and better large-map handling.

• Add export/backup options with clear privacy controls.

• Improve map package portability and reconstruction versioning.

• Expand device and environmental testing.

Phase 3 — Advanced navigation

• Explore optional route graphs and route assistance only where the extracted geometry is sufficiently reliable.

• Add richer elevation/profile information where a trustworthy source exists.

• Consider group/guide workflows and user-initiated map sharing.

• Add offline reference-data integrations only if licensing and product needs justify them.

Phase 4 — Optional 3D

• Add a 3D visualisation mode after the 2D engine is reliable.

• Distinguish illustrated map art from actual terrain geometry.

- Use known elevation data or a clearly labelled approximation; do not infer exact terrain from contour art without validation.
- Keep 3D optional and ensure the 2D view remains available for clarity, speed and battery life.

Release gates

1. Users can create and reopen a map without losing the source image.
2. Visual reconstruction preserves the source style sufficiently for the supported initial map corpus.
3. Calibration status is understandable and measured.
4. GPS position and track overlay correctly on both reconstructed and source-image views within the tested conditions.
5. Core navigation works without mobile data.
6. Tracking survives normal backgrounding and recoverable interruptions within platform constraints.
7. Privacy, battery, crash recovery and safety behaviours pass defined acceptance tests.

16. Risks and mitigations

Risk	Impact	Mitigation
Arbitrary maps are too diverse for perfect automatic reconstruction	Core value becomes inconsistent.	Start with a representative corpus, use hybrid rendering, show confidence and support corrections.
Map has no geographic references	GPS cannot be reliably aligned across the full map.	Offer calibration options; explain limitations; allow image-space fallback without overstating accuracy.
A detected decorative line is mistaken for a trail	Potentially unsafe interpretation.	Use confidence/review states; do not infer safe routes from visual resemblance alone.
Visual reconstruction changes important details	Loss of trust and navigation errors.	Preserve source assets, compare against source, use traceable feature provenance and fallback.
Background location is stopped by OS or device settings	Track gaps.	Follow platform rules, communicate tracking status, test device variations and recover gracefully.
Battery drain on long treks	App becomes unusable during trip.	Precompute reconstruction, tune GPS cadence, profile power and avoid unnecessary sensors.
Large image overwhelms memory	Crash or failed map creation.	Tiled/downsampled processing, bounded memory, checkpoints and graceful failure.
Users interpret the dot as exact	False confidence.	Show accuracy/age, calibration quality and clear warnings.
Cross-platform behaviour diverges	Inconsistent reliability.	Shared data specification, platform adapters and a common conformance test suite.
3D work delays the core product	MVP slips.	Keep 3D explicitly out of MVP and prioritise dependable 2D navigation.
Map licensing/copyright uncertainty	Legal or distribution risk.	Make ownership and lawful-use responsibilities clear; assess map import/export and sharing policy before launch.

Risk prioritisation

The highest-priority risks are map georeferencing, false confidence in extracted features, background tracking reliability and reconstruction fidelity. These should be tested early, before investing heavily in polished UI or 3D.

17. Success metrics

North-star outcome

A user can create a usable map from their own source, understand its calibration quality, and complete an offline trek while retaining access to the original map.

Proposed metrics

Metric	What it tells us
Map creation completion rate	Whether capture/import and reconstruction are understandable.
Time to first usable map	How quickly a user reaches a map they can review and save.
Calibration completion rate	Whether users can align maps without expert knowledge.
Calibration error on reference corpus	Whether geographic alignment is actually accurate.
Feature correction rate	How often automatic extraction needs user intervention.
Visual fidelity rating	Whether the reconstructed map still feels like the original.
Offline session success rate	Whether core use works with connectivity disabled.
Track continuity	Frequency and duration of unexplained track gaps under tested conditions.
Crash-free trek sessions	Reliability during the most important workflow.
Battery use per hour	Practical viability for long outdoor use.
Fallback usage and resolution	Whether the source image helps users recover from reconstruction limitations.
User trust rating	Whether uncertainty and location quality are communicated clearly.

Metrics should be segmented by map type, image quality, device tier, calibration method and environment. A single overall average can hide severe failures on a particular class of maps.

18. Open questions and decisions

These questions should be answered as the plan becomes a product specification. The current vision establishes direction but does not resolve every implementation detail.

Question	Why it matters	Suggested next step
Which map formats are in the first release?	Camera photos are different from scans, screenshots and PDF pages.	Define supported input formats and limits.
How much automatic reconstruction is required for MVP?	Perfect arbitrary-map support is not a realistic initial acceptance criterion.	Choose a supported map corpus and measurable quality bar.
What counts as sufficient calibration?	Determines when the app can confidently show a GPS dot on the map.	Define accepted reference inputs and error thresholds.
Should the app support maps with no scale or coordinates?	Such maps may only support approximate image-space positioning.	Offer a clearly labelled limited mode or require more references.
Should feature corrections be manual, assisted or both?	Affects complexity and usability.	Prototype correction UI with first-time users.
Should maps be exportable to other devices?	Raises package compatibility, privacy and sharing questions.	Decide after local package format is stable.

Question	Why it matters	Suggested next step
Is any cloud processing allowed?	Changes privacy, cost, connectivity and architecture.	Keep it out of the default path; decide only if on-device tests justify it.
What are the minimum supported phone specifications?	Sets performance and support boundaries.	Benchmark and publish a realistic device matrix.
Should the app show heading-up, north-up or both?	Affects outdoor usability and compass reliance.	Test both in a field usability study.
What 3D experience is desired later?	Illustrated 3D and realistic terrain are different products.	Keep open until 2D map representation is proven.
What is the intended business model?	Affects distribution, support and future sync features.	Decide separately from the core technical feasibility.

Recommended immediate planning activity

Before choosing the final stack, define an acceptance-test corpus of real map images and build a small proof of concept for the riskiest elements: style-preserving reconstruction, geographic calibration, and stable GPS overlay. This is a planning recommendation, not a request to begin implementation now.

19. Glossary

Term	Plain-language definition
Georeferencing	Connecting positions on an image or map to real-world geographic coordinates.
Calibration	The process of establishing how map positions correspond to known locations.
Coordinate transform	A mathematical mapping between pixel coordinates, map coordinates and geographic coordinates.
OCR	Optical character recognition: software that reads text in images.
Computer vision	Techniques for extracting useful information from images.
Vector geometry	Points, lines and shapes represented mathematically rather than as fixed pixels.
Raster image	An image made from pixels, such as a photograph or scan.
Semantic feature	An object with a meaning, such as a trail, river, label, campsite or contour line.
Breadcrumb track	A sequence of recorded GPS positions showing where the user has travelled.
Confidence score	An estimate of how likely a detection or interpretation is to be correct; not a guarantee.
Offline-first	A system designed to function without a network connection for its essential tasks.
Hybrid rendering	Combining original image texture with vector objects and digital overlays.

Closing summary

The app’s differentiator is the combination of three capabilities: preserve the source map’s visual identity, create a structured digital representation, and align live GPS movement to it while offline. The most credible route to this product is to prove those capabilities in 2D first, retain the original map as a failsafe, and make uncertainty explicit whenever the source map or GPS does not support reliable alignment.

> **Guiding rule: the reconstructed map should look like the map the user brought, but the app must never confuse visual resemblance with geographic accuracy.**